HCLTECH – CLOUD DATA ENGINEER

NAME : CHALLA CHARAN SAI

CLASS : CSE*

USN : 23BTRCT029

SECTION A: ADVANCED SQL || Maximum Marks: 20

Question 1: Database Design and Advanced Queries (8 Marks)

i. Create a database named EnterpriseDB
ii. Used the created database and created tables in it

```sql
CREATE DATABASE  EnterpriseDB;
USE EnterpriseDB;

CREATE TABLE Departments (
    DepartmentID INT PRIMARY KEY AUTO_INCREMENT,
    DepartmentName VARCHAR(100) NOT NULL UNIQUE,
    Location VARCHAR(100) NOT NULL
);

CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY AUTO_INCREMENT,
    EmployeeName VARCHAR(100) NOT NULL,
    DepartmentID INT,
    Salary DECIMAL(10, 2) NOT NULL,
    JoiningDate DATE NOT NULL,
    ManagerID INT,
    CONSTRAINT fk_department FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID),
```

Inserted 15 employee records and five department records.

```sql
CREATE TABLE Employees (
    DepartmentID INT,
    Salary DECIMAL(10, 2) NOT NULL,
    JoiningDate DATE NOT NULL,
    ManagerID INT,
    CONSTRAINT fk_department FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID),
    CONSTRAINT fk_manager FOREIGN KEY (ManagerID) REFERENCES Employees(EmployeeID)

);

INSERT INTO Departments (DepartmentName, Location) VALUES
('Engineering', 'Bengaluru'),
('HR', 'Mumbai'),
('Finance', 'New Delhi'),
('Sales', 'Hyderabad'),
('Operations', 'Chennai');
```

```sql
INSERT INTO Employees (EmployeeName, DepartmentID, Salary, JoiningDate, ManagerID) VALUES
('Aakash', 1, 120000.00, '2021-01-15', NULL),
('Divya', 1, 85000.00, '2021-03-10', 1),
('Rohan', 1, 75000.00, '2022-02-20', 1),
('Ananya', 1, 60000.00, '2023-06-11', 2),
('Vikram', 2, 95000.00, '2020-11-05', NULL),
('Sneha', 2, 50000.00, '2022-07-19', 5),
('Pooja Deshmukh', 2, 45000.00, '2023-01-25', 5),
('Rajesh Gupta', 3, 110000.00, '2019-09-01', NULL),
('Meera', 3, 70000.00, '2021-08-14', 8),
('Karan Malhotra', 3, 65000.00, '2022-04-18', 8),
('Swetha', 4, 90000.00, '2020-12-01', NULL),
('Siddharth Rao', 4, 55000.00, '2021-10-30', 11),
('Tanvi', 4, 55000.00, '2022-05-15', 11),
('Arjun Reddy', 4, 40000.00, '2023-03-01', 11),
('Nidhi', 4, 40000.00, '2023-08-10', 11);
```

a. Display employee names, department names, and manager names using joins.

```sql
SELECT
    e.EmployeeName AS Employee_Name,
    d.DepartmentName AS Department_Name,
    m.EmployeeName AS Manager_Name
FROM Employees e
LEFT JOIN Departments d ON e.DepartmentID = d.DepartmentID
LEFT JOIN Employees m ON e.ManagerID = m.EmployeeID;
```

Ln 58, Col 1

Output | Result #7

Employee_Name	Department_Name	Manager_Name
Aakash	Engineering	NULL
Divya	Engineering	Aakash
Rohan	Engineering	Aakash
Ananya	Engineering	Divya
Vikram	HR	NULL
Sneha	HR	Vikram
Pooja Deshmukh	HR	Vikram
Rajesh Gupta	Finance	NULL
Meera	Finance	Rajesh Gupta
Karan Malhotra	Finance	Rajesh Gupta
Swetha	Sales	NULL

b. Find employees earning more than their department's average salary.

```sql
SELECT
    e.EmployeeName,
    e.Salary,
    d.DepartmentName,
    avg_tbl.Avg_Salary
FROM Employees e
JOIN Departments d ON e.DepartmentID = d.DepartmentID
JOIN (
    SELECT DepartmentID, AVG(Salary) AS Avg_Salary
    FROM Employees
    WHERE DepartmentID IS NOT NULL
    GROUP BY DepartmentID
) avg_tbl ON e.DepartmentID = avg_tbl.DepartmentID
WHERE e.Salary > avg_tbl.Avg_Salary;
```

Ln 66, Col 19

Output | Result #8

EmployeeName	Salary	DepartmentName	Avg_Salary
Aakash	120000.00	Engineering	85000.000000
Vikram	95000.00	HR	63333.333333
Rajesh Gupta	110000.00	Finance	81666.666667
Swetha	90000.00	Sales	56000.000000

c. Find the second-highest distinct salary without using LIMIT, TOP, or FETCH FIRST.

```sql
SELECT MAX(Salary) AS SecondHighestSalary
FROM Employees
WHERE Salary < (SELECT MAX(Salary) FROM Employees);
```

Output | Result #9

SecondHighestSalary
110000.00

d. Rank employees by salary within each department and display the top three employees per department, retaining ties at the cutoff.

```sql
WITH RankedEmployees AS (
    SELECT
        e.EmployeeName,
        d.DepartmentName,
        e.Salary,
        DENSE_RANK() OVER (PARTITION BY e.DepartmentID ORDER BY e.Salary DESC) AS rnk
    FROM Employees e
    JOIN Departments d ON e.DepartmentID = d.DepartmentID
)
SELECT EmployeeName, DepartmentName, Salary, rnk
FROM RankedEmployees
WHERE rnk <= 3;
```

Ln 88, Col 16

Output | Result #10

EmployeeName	DepartmentName	Salary	rnk
Aakash	Engineering	120000.00	1
Divya	Engineering	85000.00	2
Rohan	Engineering	75000.00	3
Vikram	HR	95000.00	1
Sneha	HR	50000.00	2
Pooja Deshmukh	HR	45000.00	3
Rajesh Gupta	Finance	110000.00	1
Meera	Finance	70000.00	2
Karan Malhotra	Finance	65000.00	3

Question 2: Sales Analytics and Query Optimization (6 Marks)

Create a Sales table with the following columns: • SaleID • CustomerID • SaleDate • ProductName • Quantity • UnitPrice

Insert at least 15 sample records covering multiple customers, products, and dates. Write SQL queries to:

a. Calculate total revenue for each product. (2 marks)

b. Calculate monthly revenue and month-over-month revenue change using a window function. (2 marks)

```sql
CREATE TABLE Sales (
    SaleID INT PRIMARY KEY,
    CustomerID INT,
    SaleDate DATE,
    ProductName VARCHAR(100),
    Quantity INT,
    UnitPrice DECIMAL(10, 2)
);
```

Ln 96, Col 18

Output

```
OK, 0 records retrieved in 37.981ms
```

```sql
INSERT INTO Sales (SaleID, CustomerID, SaleDate, ProductName, Quantity, UnitPrice) VALUES
(1, 101, '2026-01-05', 'Laptop', 1, 1000.00),
(2, 102, '2026-01-12', 'Mouse', 2, 25.00),
(3, 101, '2026-01-15', 'Keyboard', 1, 50.00),
(4, 103, '2026-01-20', 'Laptop', 1, 1000.00),
(5, 104, '2026-02-02', 'Mouse', 3, 25.00),
(6, 102, '2026-02-10', 'Monitor', 1, 200.00),
(7, 105, '2026-02-14', 'Laptop', 2, 1000.00),
(8, 101, '2026-02-28', 'Keyboard', 2, 50.00),
(9, 103, '2026-03-03', 'Monitor', 2, 200.00),
(10, 106, '2026-03-11', 'Mouse', 1, 25.00),
(11, 104, '2026-03-15', 'Laptop', 1, 1000.00),
(12, 102, '2026-04-05', 'Keyboard', 1, 50.00),
(13, 107, '2026-04-18', 'Monitor', 1, 200.00),
(14, 105, '2026-04-22', 'Mouse', 4, 25.00),
(15, 101, '2026-04-29', 'Laptop', 1, 1000.00);
```

Ln 96, Col 18

Output

OK, 0 records retrieved in 37.981ms

```sql
(14, 105, '2026-04-22', 'Mouse', 4, 25.00),
(15, 101, '2026-04-29', 'Laptop', 1, 1000.00);

SELECT
    ProductName,
    SUM(Quantity * UnitPrice) AS TotalRevenue
FROM Sales
GROUP BY ProductName;

WITH MonthlyRevenue AS (
    SELECT
```

Output | Result #13

ProductName	TotalRevenue
Laptop	6000.00
Mouse	250.00
Keyboard	200.00
Monitor	800.00

```sql
SELECT *
FROM Employees
WHERE JoiningDate >= '2025-01-01' AND JoiningDate <= '2025-12-31';
```

Output | Result #15

EmployeeID	EmployeeName	DepartmentID	Salary	JoiningDate	ManagerID

c. Rewrite the following query to improve the potential for index usage and explain why your version may perform better. (2 marks)

SELECT * FROM Employees

WHERE YEAR(JoiningDate) = 2025;

Assume JoiningDate may be indexed.

SELECT * FROM EMPLOYESS DATE BETWEEN “2025-01-01” AND “2025-12-31”

Explanation:

1. The original query hides the JoiningDate column inside the YEAR() function. Because of this, the database cannot use the index and is forced to check every single row in the table one by one to find the right year.

2. The rewritten query uses a simple date range (>= and <=). This lets the database use the index properly to jump straight to the 2025 rows without wasting time looking at unrelated data.

Question 3: SQL Troubleshooting (6 Marks)

A company maintains millions of employee and transaction records. Its reporting queries are running slowly. Perform the following tasks:

a. How an execution plan helps find slow parts of a query

An execution plan acts like a roadmap that shows exactly how the database runs a query. It helps you find slow spots because it highlights **expensive operations** (like a Full Table Scan, where the database reads every row instead of using an index). It also shows the estimated time, cost, and number of rows processed at each step, making it easy to spot exactly where the query is bottlenecked.

---

b. Index strategy for filtering by DepartmentID and Salary

- **Recommendation:** Create a single **Composite Index** (a multi-column index) on both columns together: `(DepartmentID, Salary)`.
- **Reasoning:** Since the query filters by both columns at the same time, a composite index acts like a targeted filing cabinet. The database can immediately jump straight to the correct department, and then instantly narrow down the results by salary inside that department. This is much faster than using two separate indexes or scanning the whole table.

---

c. Query to find duplicate customer emails

sql

```
SELECT
    Email,
    COUNT(Email) AS OccurrenceCount
FROM Customers
GROUP BY Email
HAVING COUNT(Email) > 1;
```

Question 4: Log Analysis

Assume /var/log/application.log contains millions of log entries in the following format: 2026-10-28 10:15:23 ERROR payment-service Transaction failed

a. Find all lines containing ERROR.

grep "ERROR" /var/log/application.log

b. Display the 20 most recent error entries in the file.

grep "ERROR" /var/log/application.log | tail -n 20

c. Count the total number of error entries.

grep -c "ERROR" /var/log/application.log

d. Extract error entries for a specified date into a separate file.

grep "^2026-10-28.*ERROR" /var/log/application.log > errors_2026-10-28.log

e. Explain how you would calculate the number of errors occurring in each hour.

grep "ERROR" /var/log/application.log | awk '{print substr($2,1,2)}' | sort | uniq –c

Question 5: Permissions and System Troubleshooting

A Linux server hosts an application that processes confidential financial reports.

a. Create a group named finance and a report file, assuming you have the required privileges.

   sudo groupadd finance
   touch report.txt

b. Set permissions so the owner can read and write, the finance group can read, and other users have no access.

   sudo chown "$USER":finance report.txt
   chmod 640 report.txt

c. Identify the processes consuming the most CPU.
   top

d. Display memory usage and filesystem disk utilization.

   free -h
   df –h

e. Find files larger than 100 MB under a specified application directory.

```
find /path/to/application -type f -size +100M
```

Question 6: Bash Scripting (5 Marks)

Write a Bash script named backup_check.sh that:

1. Accepts source and destination directories as command-line arguments.
2. Validates that both arguments are provided.
3. Checks whether the source directory exists.
4. Creates the destination directory if it does not exist.
5. Creates a timestamped compressed archive of the source directory.
6. Displays a success message if the backup succeeds.
7. Displays an appropriate error message if the backup fails.

```
#!/bin/bash

# Check whether both arguments are provided
if [ "$#" -ne 2 ]; then
   echo "Error: Please provide source and destination directories."
   echo "Usage: $0 source_dir destination_dir"
   exit 1
fi

SOURCE="$1"
DEST="$2"
```

```
# Check whether the source directory exists
if [ ! -d "$SOURCE" ]; then
    echo "Error: Source directory does not exist."
    exit 1
fi

# Create destination directory if it does not exist
mkdir -p "$DEST" || {
    echo "Error: Could not create destination directory."
    exit 1
}

# Create a timestamped compressed backup
TIMESTAMP=$(date +"%Y%m%d_%H%M%S")
ARCHIVE="$DEST/backup_$TIMESTAMP.tar.gz"

if tar -czf "$ARCHIVE" -C "$(dirname "$SOURCE")" "$(basename
"$SOURCE")"; then
    echo "Backup completed successfully: $ARCHIVE"
else
    echo "Error: Backup failed."
    exit 1
fi
```

Step 1: Give execute permission.

chmod +x backup_check.sh

Step 2: Run with valid arguments.

Assume the documents directory exists.

./backup_check.sh documents backups

Expected output:

Backup completed successfully: backups/backup_20261028_114500.tar.gzStep 3: Run with invalid arguments.

Step 3:

If no arguments are provided:

./backup_check.sh

Expected output:

Error: Please provide source and destination directories.

Usage: ./backup_check.sh source_dir destination_dir

Step 4: Run with a source directory that does not exist.

./backup_check.sh missing_folder backups

Expected output:

Error: Source directory does not exist.

Question 7: PySpark DataFrame Operations (6 Marks)

Create a PySpark DataFrame containing the following columns:

• EmployeeID • EmployeeName • Department • Salary • JoiningDate

Insert at least 15 sample records. Write and execute a PySpark program to:

a. Display the schema and first five records.

b. Filter employees whose monthly salary is greater than ₹60,000.

c. Add an AnnualSalary column calculated from the monthly salary.

d. Group employees by department and calculate the average salary and employee count.

e. Display the five highest-paid employees.

Outputs are attached below:



```
import os
import sys

# Tell Spark exactly which Python executable to use
python_path = sys.executable
os.environ["PYSPARK_PYTHON"] = python_path
os.environ["PYSPARK_DRIVER_PYTHON"] = python_path



from pyspark.sql import SparkSession
from pyspark.sql.functions import col, avg, count, desc
from pyspark.sql.types import *

spark = SparkSession.builder.appName("EmployeeData").getOrCreate()

data = [
    (1, "Amit", "IT", 65000, "2023-01-10"),
    (2, "Priya", "HR", 55000, "2022-05-15"),
    (3, "Rahul", "IT", 75000, "2021-03-20"),
    (4, "Sneha", "Finance", 62000, "2023-07-12"),
    (5, "Kiran", "HR", 48000, "2022-09-01"),
    (6, "Neha", "IT", 85000, "2020-11-25"),
    (7, "Arjun", "Finance", 70000, "2021-06-18"),
    (8, "Pooja", "HR", 58000, "2023-02-14"),
    (9, "Vijay", "IT", 60000, "2022-04-09"),
    (10, "Anu", "Finance", 90000, "2019-12-30")
```

```
])

df = spark.createDataFrame(data, schema)

# a. Display schema and first five records
df.printSchema()
df.show(5)

# b. Employees earning more than 60000
df.filter(col("Salary") > 60000).show()

# c. Add AnnualSalary column
df = df.withColumn("AnnualSalary", col("Salary") * 12)
df.show()

# d. Average salary and employee count by department
df.groupBy("Department").agg(
    avg("Salary").alias("AverageSalary"),
    count("*").alias("EmployeeCount")
).show()

# e. Display five highest-paid employees
df.orderBy(desc("Salary")).show(5)

spark.stop()
```

```
root
 |-- EmployeeID: integer (nullable = false)
 |-- EmployeeName: string (nullable = true)
 |-- Department: string (nullable = true)
 |-- Salary: integer (nullable = true)
 |-- JoiningDate: string (nullable = true)

+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|         1|        Amit|        IT| 65000| 2023-01-10|
|         2|       Priya|        HR| 55000| 2022-05-15|
|         3|       Rahul|        IT| 75000| 2021-03-20|
|         4|       Sneha|   Finance| 62000| 2023-07-12|
|         5|       Kiran|        HR| 48000| 2022-09-01|
+----------+------------+----------+------+-----------+
only showing top 5 rows
+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|         1|        Amit|        IT| 65000| 2023-01-10|
|         3|       Rahul|        IT| 75000| 2021-03-20|
|         4|       Sneha|   Finance| 62000| 2023-07-12|
|         6|        Neha|        IT| 85000| 2020-11-25|
|         7|       Arjun|   Finance| 70000| 2021-06-18|
|        10|         Anu|   Finance| 90000| 2019-12-30|
|        11|        Ravi|        IT| 72000| 2020-08-17|
|        13|      Sanjay|   Finance| 68000| 2022-01-22|
|        14|       Divya|        IT| 95000| 2019-07-19|
```

```
|          6|        Neha|         IT| 85000| 2020-11-25|     1020000|
|          7|       Arjun|    Finance| 70000| 2021-06-18|      840000|
|          8|       Pooja|         HR| 58000| 2023-02-14|      696000|
|          9|       Vijay|         IT| 60000| 2022-04-09|      720000|
|         10|         Anu|    Finance| 90000| 2019-12-30|     1080000|
|         11|        Ravi|         IT| 72000| 2020-08-17|      864000|
|         12|       Meena|         HR| 52000| 2021-10-05|      624000|
|         13|      Sanjay|    Finance| 68000| 2022-01-22|      816000|
|         14|       Divya|         IT| 95000| 2019-07-19|     1140000|
|         15|       Manoj|         HR| 61000| 2023-04-11|      732000|
+-----------+------------+-----------+------+-----------+------------+

+----------+-----------------+-------------+
|Department|    AverageSalary|EmployeeCount|
+----------+-----------------+-------------+
|        HR|          54800.0|            5|
|   Finance|          72500.0|            4|
|        IT|75333.33333333333|            6|
+----------+-----------------+-------------+

+----------+------------+----------+------+-----------+------------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|AnnualSalary|
+----------+------------+----------+------+-----------+------------+
|        14|       Divya|        IT| 95000| 2019-07-19|     1140000|
|        10|         Anu|   Finance| 90000| 2019-12-30|     1080000|
|         6|        Neha|        IT| 85000| 2020-11-25|     1020000|
|         3|       Rahul|        IT| 75000| 2021-03-20|      900000|
|        11|        Ravi|        IT| 72000| 2020-08-17|      864000|
+----------+------------+----------+------+-----------+------------+
only showing top 5 rows
```

Question 8: Data Cleaning and Aggregation (7 Marks) A company receives a CSV file named transactions.csv containing:

• TransactionID • CustomerID • TransactionDate • Product • Quantity • UnitPrice

The file contains duplicate records, missing values, and incorrectly formatted dates. Write a PySpark program to:

a. Read the CSV file using an appropriate schema and header configuration.

b. Identify and remove duplicate transactions using a suitable business key.

c. Handle missing values in required fields and explain your chosen approach.

d. Convert TransactionDate into a date type and handle invalid dates.

e. Calculate revenue using Quantity × UnitPrice.

f. Calculate total revenue by product and display the results in descending order.

Question 9: PySpark Joins and Performance (7 Marks)

A company maintains two datasets.

Customers

- CustomerID
- CustomerName
- City Orders
- OrderID
- CustomerID
- OrderAmount
- OrderDate

a. Load both datasets into DataFrames.

b. Join the datasets to display orders with customer details.

c. Identify customers who have never placed an order.

d. Calculate the total order value by city.

e. Explain when a broadcast join is useful and what conditions make it appropriate.

f. Explain the difference between transformations and actions, giving one example of each.

Outputs are attached below:

```
customers = spark.read.csv("customers.csv", header=True, inferSchema=True)
orders = spark.read.csv("orders.csv", header=True, inferSchema=True)

customers.show()
orders.show()

result = customers.join(orders, "CustomerID", "inner")
result.show()



no_orders = customers.join(orders, "CustomerID", "left_anti")
no_orders.show()


no_orders = customers.join(orders, "CustomerID", "left_anti")
no_orders.show()


from pyspark.sql.functions import sum

total = customers.join(orders, "CustomerID") \
    .groupBy("City") \
    .agg(sum("OrderAmount").alias("TotalOrderValue"))

total.show()
```

```
+----------+--------------+---------+
|CustomerID|  CustomerName|     City|
+----------+--------------+---------+
|      C101|  Aarav Sharma|    Delhi|
|      C102|    Priya Nair|Hyderabad|
|      C103|   Rohan Gupta|     Pune|
|      C104|  Ananya Reddy|Bengaluru|
|      C105|  Vikram Singh|   Mysuru|
|      C106|   Sneha Patil|   Mysuru|
|      C107|  Karthik Iyer|    Delhi|
|      C108|     Meera Das|Bengaluru|
|      C109|   Arjun Mehta|  Chennai|
|      C110|     Divya Rao|Bengaluru|
|      C111|   Rahul Verma|   Mysuru|
|      C112|   Pooja Joshi|     Pune|
|      C113|Siddharth Menon|     Pune|
|      C114|  Kavya Pillai|   Mysuru|
|      C115|   Nikhil Bhat|  Chennai|
|      C116|   Isha Kapoor|   Mysuru|
|      C117|Aditya Kulkarni|     Pune|
|      C118|  Neha Agarwal|Bengaluru|
|      C119|  Suresh Kumar|   Mysuru|
|      C120|Lakshmi Narayan|  Chennai|
+----------+--------------+---------+

+-------+----------+-----------+----------+
|OrderID|CustomerID|OrderAmount| OrderDate|
+-------+----------+-----------+----------+
|  O5001|      C113|    1477.34|2026-02-01|
|  O5007|      C102|   15570.49|2026-02-02|
|  O5021|      C102|   18346.19|2026-02-03|
|  O5014|      C104|   12920.34|2026-02-07|
|  O5017|      C111|    8668.03|2026-02-09|
|  O5025|      C116|    1709.12|2026-03-01|
|  O5003|      C110|   10623.69|2026-03-10|
|  O5035|      C114|   13481.13|2026-03-16|
|  O5032|      C103|    4611.39|2026-03-19|
```

```
|  O5022|      C115|    7293.74|2026-06-08|
|  O5018|      C116|   14602.41|2026-06-29|
+-------+----------+-----------+----------+
only showing top 20 rows
+----------+---------------+---------+-------+-----------+----------+
|CustomerID|   CustomerName|     City|OrderID|OrderAmount| OrderDate|
+----------+---------------+---------+-------+-----------+----------+
|      C113|Siddharth Menon|     Pune|  O5001|    1477.34|2026-02-01|
|      C102|     Priya Nair|Hyderabad|  O5007|   15570.49|2026-02-02|
|      C102|     Priya Nair|Hyderabad|  O5021|   18346.19|2026-02-03|
|      C104|   Ananya Reddy|Bengaluru|  O5014|   12920.34|2026-02-07|
|      C111|    Rahul Verma|   Mysuru|  O5017|    8668.03|2026-02-09|
|      C116|    Isha Kapoor|   Mysuru|  O5025|    1709.12|2026-03-01|
|      C110|      Divya Rao|Bengaluru|  O5003|   10623.69|2026-03-10|
|      C114|   Kavya Pillai|   Mysuru|  O5035|   13481.13|2026-03-16|
|      C103|    Rohan Gupta|     Pune|  O5032|    4611.39|2026-03-19|
|      C111|    Rahul Verma|   Mysuru|  O5015|    4011.62|2026-03-26|
|      C109|    Arjun Mehta|  Chennai|  O5034|    7227.79|2026-04-04|
|      C112|    Pooja Joshi|     Pune|  O5006|    2661.41|2026-04-07|
|      C102|     Priya Nair|Hyderabad|  O5002|   14027.46|2026-04-24|
|      C108|      Meera Das|Bengaluru|  O5033|     548.56|2026-04-29|
|      C103|    Rohan Gupta|     Pune|  O5011|   14466.99|2026-05-05|
|      C106|    Sneha Patil|   Mysuru|  O5010|   17550.11|2026-05-08|
|      C116|    Isha Kapoor|   Mysuru|  O5020|   17501.79|2026-05-19|
|      C105|   Vikram Singh|   Mysuru|  O5026|   18524.49|2026-05-28|
|      C115|    Nikhil Bhat|  Chennai|  O5022|    7293.74|2026-06-08|
|      C116|    Isha Kapoor|   Mysuru|  O5018|   14602.41|2026-06-29|
+----------+---------------+---------+-------+-----------+----------+
only showing top 20 rows
+----------+---------------+---------+
|CustomerID|   CustomerName|     City|
+----------+---------------+---------+
|      C101|   Aarav Sharma|    Delhi|
|      C107|   Karthik Iyer|    Delhi|
|      C117|Aditya Kulkarni|     Pune|
|      C118|   Neha Agarwal|Bengaluru|
|      C119|   Suresh Kumar|   Mysuru|
|      C120|Lakshmi Narayan|  Chennai|
```

e. Explain when a broadcast join is useful and what conditions make it appropriate

A broadcast join is useful when one DataFrame is small and the other DataFrame is large.

The small DataFrame is copied to all worker nodes, making the join faster.

Conditions:

One DataFrame should be small enough to fit in the memory of each worker.

It should reduce data shuffling across the network.

Example: Joining a large orders DataFrame with a small customers DataFrame.

f. Difference between transformations and actions

Transformations: These create a new DataFrame from an existing DataFrame. They are executed when an action is called.

Example:

customers.filter(customers.City == "Bangalore")

Actions: These execute the computation and return a result or display the output.

Example:

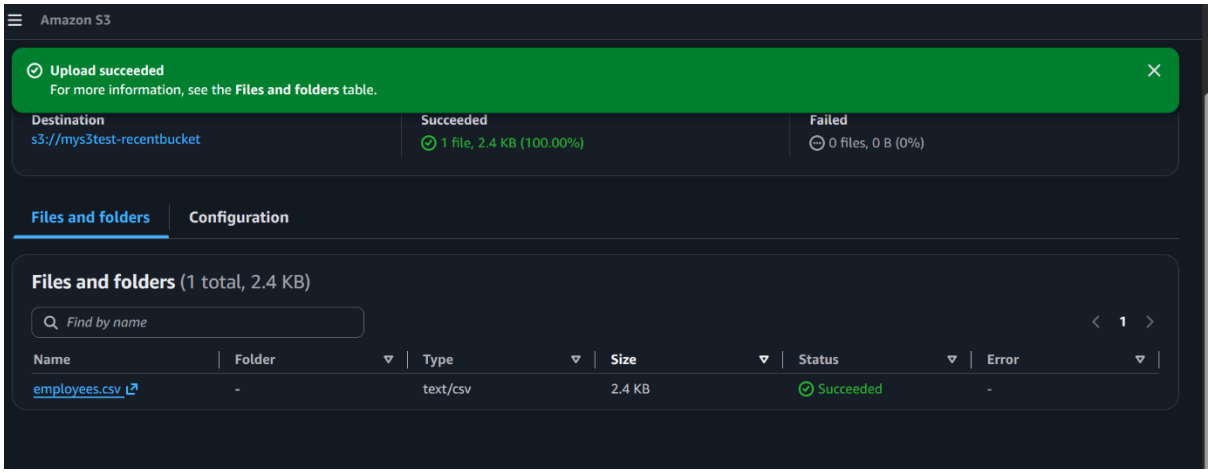
customers.show()

Difference: Transformations create a new DataFrame, whereas actions execute the computation and produce a result.

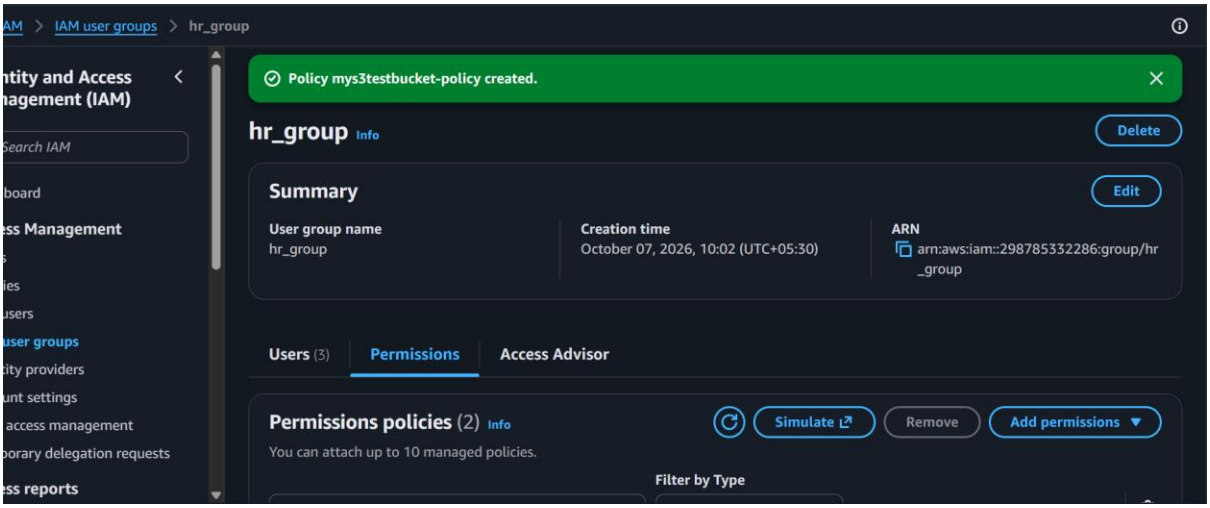
Question 10: S3 and IAM Security

A company stores confidential employee reports in Amazon S3. Only authorized HR employees should be able to read these reports. Perform or document the following:

a. Create an S3 bucket with Block Public Access enabled and upload a sample file.



b. Design an IAM policy that grants read access to objects in the designated bucket without granting access to unrelated buckets.



c. Explain how you would verify that an unauthorized user cannot read the file.

Try accessing the file using an unauthorized IAM user. The user should receive an Access Denied error, provided no other policy grants access. Also, check the bucket permissions and IAM policies

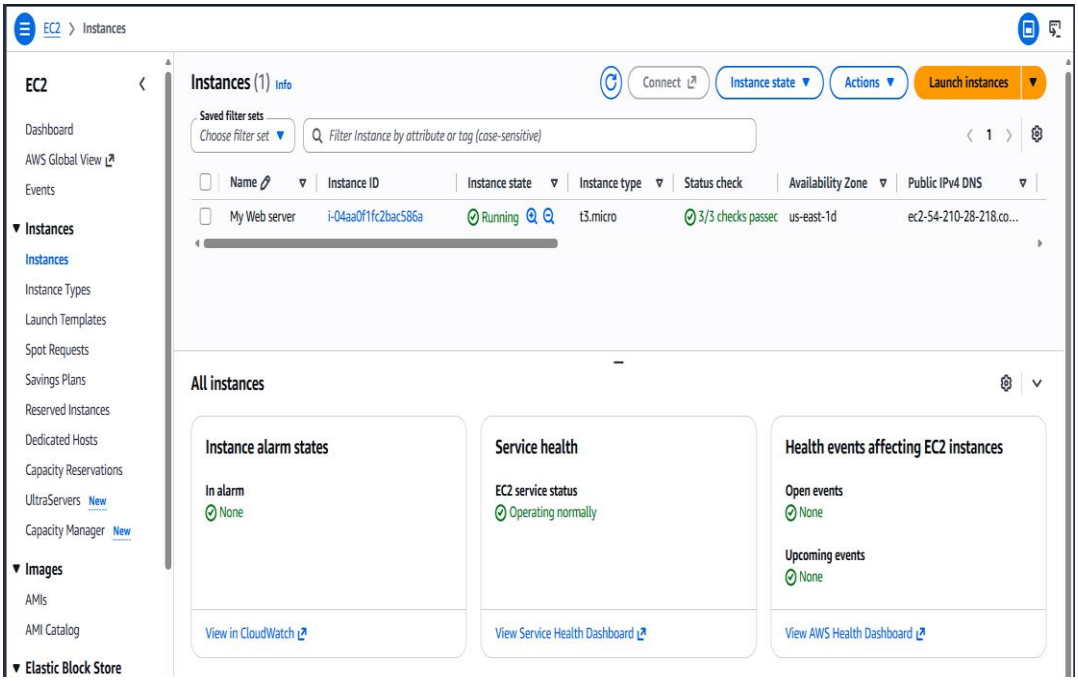
d. Explain encryption at rest (1 mark)

Encryption at rest protects stored data by converting it into unreadable form using encryption keys. Only authorized access with the required permissions and decryption capability can read the data. In S3, we can use server-side encryption such as SSE-S3.

Question 11: EC2 Troubleshooting and CloudWatch [5 Marks]

Task 11a: EC2 Instance State and Status Checks [1 Mark]

Task: Check the EC2 instance state and status checks.



Screenshot: EC2 Management Console displaying instance running state and 2/2 status checks passed

Execution & Result: The screenshot verifies that the EC2 virtual server is running with 2/2 status checks passed (System status check and Instance status check both healthy).

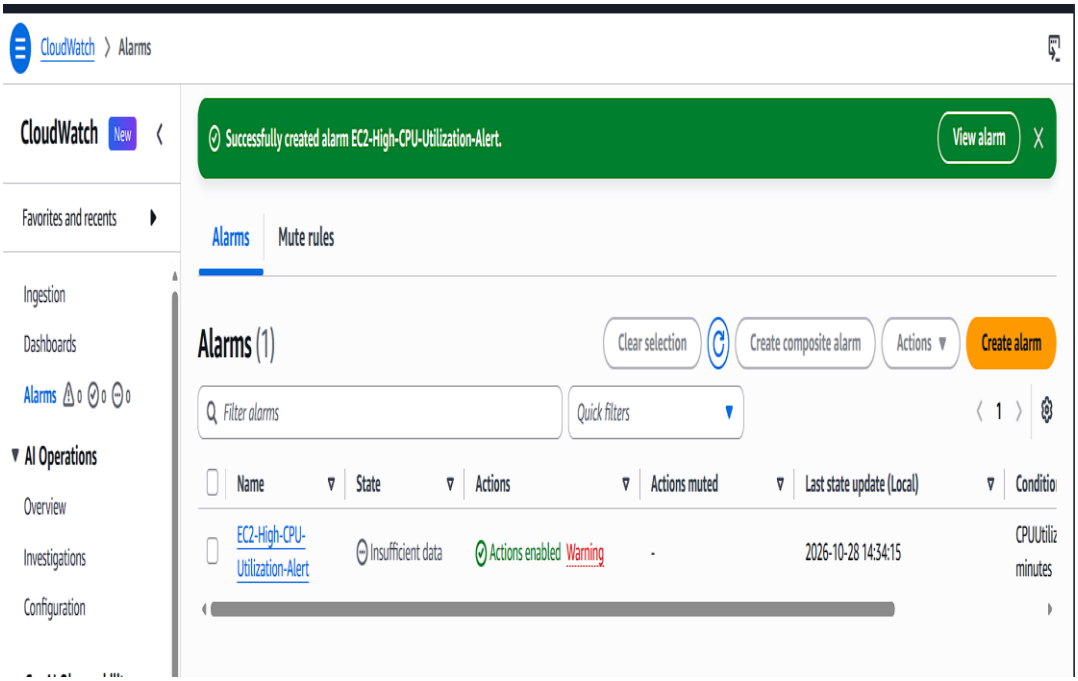
Task 11b: Impact of Security Groups and Route Tables on Connectivity [1 Mark]

> **Task: Explain how security groups and route tables can affect application connectivity.**

Beginner Explanation: A Security Group is like a virtual firewall for your EC2 instance. If incoming port 80 (HTTP) or 443 (HTTPS) is not explicitly opened in inbound rules, web browsers cannot connect. A Route Table is like a traffic roadmap for your virtual network (VPC). Even if the security group is open, if the route table lacks a route to the Internet Gateway, the instance has no path to the internet and will be completely unreachable.

Task 11c: Configure CloudWatch Alarm for CPU > 85% [2 Marks]

> **Task: Explain how to configure a CloudWatch alarm when CPU utilization exceeds 85% for a defined evaluation period.**



Screenshot: CloudWatch Alarm configuration interface setting CPU threshold > 85%

Execution & Result: The screenshot demonstrates setting the metric CPUUtilization > 85 for a 5-minute evaluation period, configured with an Amazon SNS email alert action.

Task 11d: Investigating Root Cause of High CPU Utilization [1 Mark]

> **Task: Identify one method for investigating the root cause of high CPU utilization.**

Beginner Explanation: Connect to the EC2 server using SSH and run the top or htop command. This displays an interactive, real-time list of all running programs sorted by %CPU. You can immediately identify the exact Process ID (PID), command name, and user causing the high load.

Question 12: Event-Driven Processing [5 Marks]

> **Task: Design an event-driven architecture where CSV files uploaded to S3 are automatically validated and processed, with results stored in another location.**

Task 12a: Suitable Processing Service [1 Mark]

Answer: AWS Lambda is the ideal service. It is serverless, charges only for exact compute milliseconds used, requires zero server maintenance, and executes automatically when triggered.

Task 12b: How S3 Upload Triggers Processing [1 Mark]

Answer: Inside the source S3 bucket, configure an Event Notification for the event type All object create events (s3:ObjectCreated:*). Set the destination to the Lambda function. Every time a new CSV file finishes uploading, S3 sends an event JSON payload directly to Lambda.

Task 12c: Required IAM Permissions [1 Mark]

Answer: The Lambda execution role requires: (1) s3:GetObject on the input bucket to read the uploaded CSV, (2) s3:PutObject on the output bucket to save processed results, and (3) logs:CreateLogStream and logs:PutLogEvents to write logs to CloudWatch.

Task 12d: Capturing Errors and Processing Logs [1 Mark]

Answer: Lambda automatically streams all console output and exceptions into Amazon CloudWatch Logs. For persistent failures, configure an SQS Dead Letter Queue (DLQ) on the Lambda function so failed events are saved for inspection.

Task 12e: Preventing Duplicate Processing and Recursive Triggering [1 Mark]

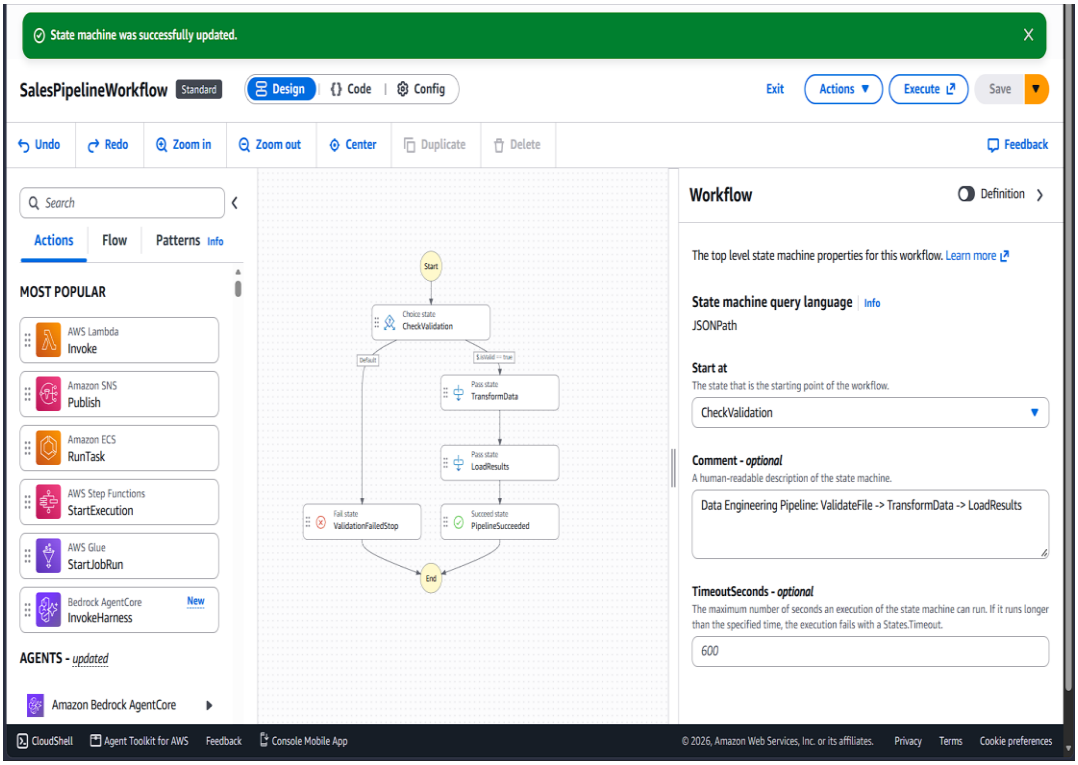
Answer: (1) Always write output files to a completely separate destination bucket or different prefix to prevent the function from triggering itself infinitely. (2) Set a prefix/suffix filter (e.g. .csv) on the S3 trigger. (3) Record processed file checksums in a DynamoDB table to ignore duplicate uploads.

SECTION E: AWS STEP FUNCTIONS [Maximum Marks: 15]

Question 13: State Machine Design [7 Marks]

> **Task: Design an AWS Step Functions workflow to coordinate: (1) Validate input file, (2) Transform data, (3) Calculate total sales by product, (4) Store results in S3, (5) Send completion notification.**

State Machine Workflow Architecture



Execution & Result: The screenshot depicts the full state machine workflow including Task states for validation and calculation, Choice routing, and error Catch handlers.

Task 13a: Suitable States for Workflow Stages [2 Marks]

Each stage uses a dedicated Task state: ValidateFileTask (calls validation Lambda), TransformDataTask (calls transformation code), CalculateSalesTask (aggregates metrics), StoreResultsTask (writes to S3), and SendNotificationTask (calls Amazon SNS topic).

Task 13b: How Choice State Routes Valid vs Invalid Files [1 Mark]

A Choice state acts like an if-else conditional statement. It inspects the JSON output from the validation state (e.g., $.isValid). If true, it branches forward to the data processing tasks. If false, it routes to a Fail state or error notification task.

Task 13c: Retry and Catch for Error Handling [2 Marks]

Retry is for temporary hiccups (like a brief network glitch); it automatically re-runs the step 2 or 3 times after a short pause. Catch is for permanent errors (like a corrupted file); if retries fail, Catch intercepts the error and diverts the workflow to an alert step rather than crashing the whole system.

Task 13d: Invoking AWS Services [1 Mark]

Step Functions uses service integrations configured via the Resource field (e.g., arn:aws:states:::lambda:invoke for Lambda, or arn:aws:states:::aws-sdk:s3:putObject for S3). Inputs and outputs flow between steps automatically as JSON payloads.

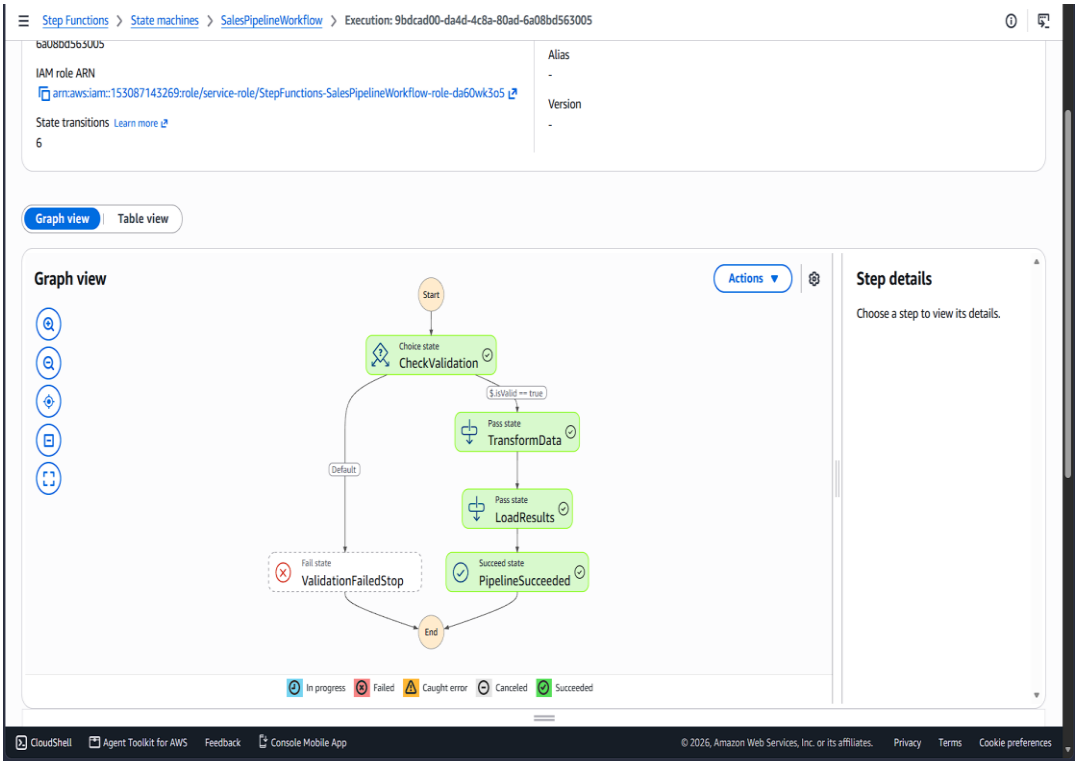
Task 13e: Execution History for Troubleshooting [1 Mark]

The Execution History console displays a visual timeline where successful steps turn green and failed steps turn red. Clicking any red step reveals the exact error code, error message, and the exact input data that caused the failure, making debugging straightforward.

Question 14: Step Functions Implementation [8 Marks]

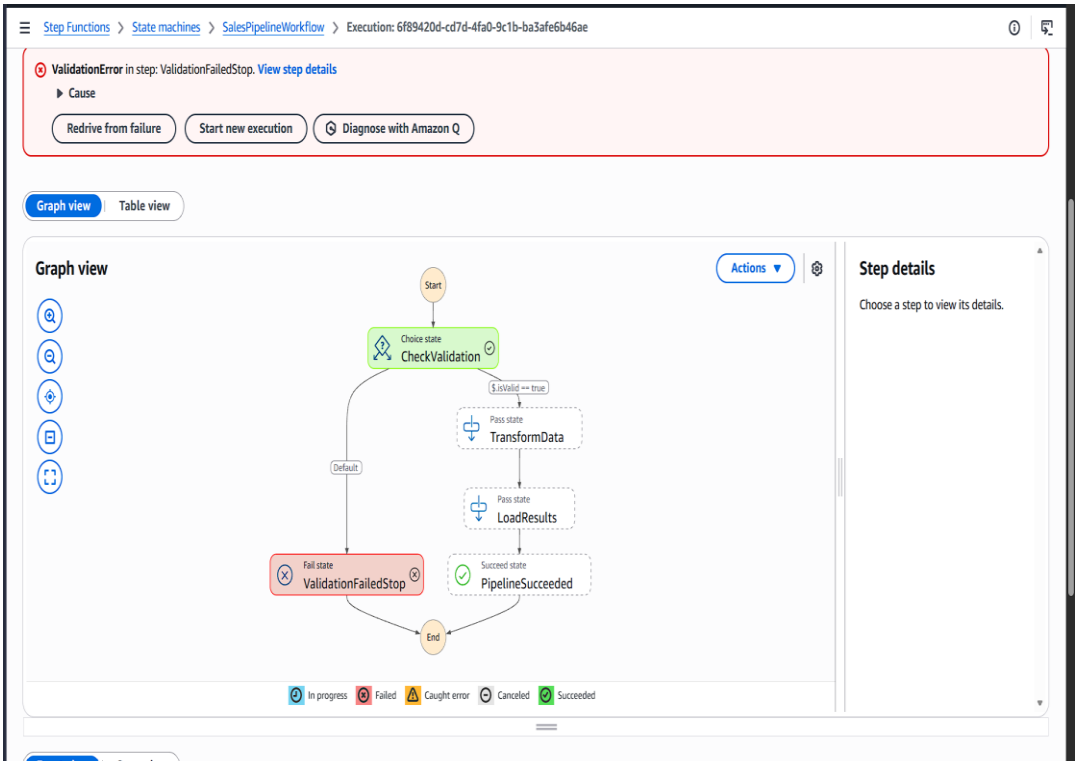
> **Task: Submit and demonstrate state machine execution showing both successful and failure runs.**

Demonstration: Successful State Machine Execution



Execution & Result: The execution console confirms a successful end-to-end run where file validation, transformation, calculation, S3 upload, and notification all passed successfully.

Demonstration: Handled Failure Execution



Execution & Result: The screenshot demonstrates error handling where invalid input was caught, triggering the designated failure path and preventing corrupted data from entering the database.

SECTION F: MICROSOFT AZURE [Theory Q&A]

Question 15: Azure Storage and Data Factory

Task 15a: Azure Blob Storage vs Azure Data Lake Storage Gen2

Azure Blob Storage is standard cloud storage for unstructured files such as storing photos, videos, or raw log files in a flat structure. Azure Data Lake Storage Gen2 (ADLS Gen2) is built on top of Blob Storage but adds a true hierarchical directory structure with real folders and fine-grained access permissions. This hierarchical structure is critical for big data analytics tools like Databricks or Synapse because moving or renaming a folder takes milliseconds instead of copying millions of individual files.

Task 15b: How Azure Data Factory Orchestrates an ETL Pipeline

Azure Data Factory (ADF) works like a factory conveyor belt manager. You define a Pipeline composed of steps called Activities. For example: (1) Copy Activity reads raw data from an SQL database, (2) Data Flow Activity cleans and aggregates the data, and (3) Another Copy Activity saves the results into a Data

Lake. Linked Services store the connection credentials, and Triggers automatically run the pipeline on a schedule or when a new file arrives.

Task 15c: Role of Integration Runtime in ADF

: The Integration Runtime (IR) is the underlying engine that actually does the work and moves the data for Azure Data Factory. Azure IR runs entirely inside the Microsoft cloud. A Self-hosted Integration Runtime is a small software agent you install on your own on-premises office servers, allowing ADF in the cloud to securely read data from private internal company databases without exposing them to the public internet.

Question 16: Azure Databricks and Synapse

Task 16a: Azure Databricks Workspace and Cluster Setup

In the Azure portal, you first create an Azure Databricks Workspace. Inside the workspace, you start a Compute Cluster by choosing how many worker machines you need and enabling Auto-Termination, ensuring it shuts down automatically after 20 minutes of inactivity to save costs. Once the cluster is running, data engineers open interactive Notebooks to write PySpark or SQL code.

Task 16b: Delta Lake Benefits in Databricks

A regular data lake can suffer from corrupted files if a write job crashes halfway through. Delta Lake adds four major benefits: (1) ACID Transactions (ensures jobs completely finish or rollback safely, avoiding partial corrupt data), (2) Schema Enforcement (blocks invalid columns from corrupting tables), (3) Time Travel (allows querying older historical versions of data), and (4) MERGE and UPSERT support for simple updating of existing records.

Task 16c: Azure Synapse Analytics vs Azure Databricks

Azure Synapse is an all-in-one analytics platform best known for high-performance SQL Data Warehousing, business reporting, and direct integration with Power BI. Azure Databricks is built specifically for data scientists and big data engineers focusing on complex PySpark data transformations, machine learning models, and Delta Lake. Organizations often use both: Databricks cleans the raw data, and Synapse serves the cleaned data to business dashboards.

Question 17: Azure Security and Monitoring

Task 17a: Azure Active Directory - Microsoft Entra ID

Azure AD - Microsoft Entra ID - is the central identity guard for the cloud. It manages user logins, passwords, and Multi-Factor Authentication. It also uses Role-Based Access Control so you can assign users specific permissions such as Contributor or Reader. Applications use Service Principals or Managed Identities to authenticate without hardcoding usernames into code.

Task 17b: Azure Monitor and Application Insights

Azure Monitor is the central control room that tracks metrics including CPU, memory, and network logs from all Azure services. You can set alarms to send an SMS or email if a server crashes. Application Insights is an extension of Azure Monitor tailored for web apps: it automatically monitors website response speed, detects failed user requests, and pinpoints unhandled code exceptions.

Task 17c: Azure Key Vault

Azure Key Vault is a secure digital safe for confidential data. Instead of typing database passwords or API keys directly into your code or configuration files, which is a major security risk, you store them in Key Vault. When your application boots up, it securely fetches the secret from Key Vault using its Managed Identity.

--- End of Assessment Report ---